

Differential Drive Mobile Robot Simulation and Dead Reckoning

Vasutorn Siriyakorn, and Mohammad Haadi Akhter

October 15, 2025

Abstract

The objective of this lab exercise was to understand how to set up the required software for the course, implement a simulation of a mobile robot equipped with sensors, and develop a dead-reckoning algorithm to localize the robot. The robot simulation designed in this lab consists of several main components, including the forward kinematics of a differential drive mobile robot and sensor simulation. For localization, the dead-reckoning algorithm was used to estimate the robot's position. This localization method relies on the calculation of the current position using the known previous position along with measurements of speed, direction, and elapsed time. In this lab, the output of the encoders on the left and right wheels was used to calculate the linear and angular velocities of the robot. The robot's velocity was obtained by converting the number of pulses counted by the encoder into the displacement of the robot's wheels, and then dividing by the encoder reading interval time. As a result, the position of the robot could be determined using the dead-reckoning localization method. However, the error between the actual position of the robot from the simulation and the output of the localization algorithm was significant due to sensor noise from the encoders.

1 Introduction

In the pre-lab session, we were taught how to estimate the next pose of the robot with respect to the world frame given the previous pose with respect to the world frame and a control command. We derived direct formulas for compounding poses so that we can obtain a formula to estimate the next pose in the world frame given a control command in the vehicle frame with the help of transformations. Also, in the pre-lab session, we were taught how to localize a robot by using sensor readings as control commands and the current pose with the help of dead reckoning approach.

In the lab session, we implemented two compounding operations - one for determining the next pose of the robot with respect to the world frame, given the control commands in the vehicle frame and the current pose with respect to the world frame. We also implemented the inverse of this operation, i.e., if we are given a pose with respect to the world frame, how do we obtain it with respect to the vehicle frame.

After this, we use these implementations in developing the forward simulation of a differential drive moving in a 2D plane having three degrees of freedom - movement along the

x-axis, movement along the y-axis and orientation in the plane with respect to the x-axis, where the counterclockwise direction from positive x-axis is taken as positive angle.

At the end, we simulate encoders and with readings from them, try to determine our pose and compare it with that given by the forward simulation of the robot

This reports is organized as follows: Section 2 describes how to simulate the robot by implementing the *DifferentialDriveSimulatedRobot* class inheriting from the *SimulatedRobot* base class. This section also explains how to use the class to simulate the trajectories requested in the lab. Section 3 details the dead reckoning localization method of a differential drive mobile robot. The technique is implemented in the *DR_3DOFDifferentialDrive* class as a child class of the *Localization* class providing an implementation for their virtual methods. Finally, the section 4 is devoted to the description of the problems founds during the lab.

2 Simulation

The simulation development begins by implementing a base class for the robot, including its state. The differential drive mobile robot has six key states which contain the pose and velocity of the robot as follow

$$\begin{aligned}\mathbf{x}_k &= \begin{bmatrix} {}^N x_k & {}^N y_k & {}^N \psi_k & {}^B u_k & {}^B v_k & {}^B r_k \end{bmatrix}^\top \\ &= \begin{bmatrix} {}^N \boldsymbol{\eta}_k & {}^B \boldsymbol{\nu}_k \end{bmatrix}^\top\end{aligned}\tag{1}$$

The pose vector at time step k , denoted as ${}^N \boldsymbol{\eta}_k$, comprises the robot's position along the x-axis (${}^N x_k$), y-axis (${}^N y_k$), and its heading, represented by the rotation about the z-axis (${}^N \psi_k$); all components are expressed in the global reference frame $\{N\}$. Correspondingly, the velocity vector at time step k , ${}^B \boldsymbol{\nu}_k$, contains the robot's linear velocity along the x-axis (${}^B u_k$), linear velocity along the y-axis (${}^B v_k$), and angular velocity about the z-axis (${}^B r_k$), with each defined in the robot reference frame $\{B\}$.

Since the pose and velocity vectors are represented in difference coordinate frames, compounding operations—which are denoted by the Oplus ($\oplus$) and Ominus ($\ominus$) operators—are employed to facilitate vector addition and subtraction across frames. These operations, implemented using the **NumPy** library, are essential for forward kinematics in robotic simulation, providing a systematic means of combining vectors formulated in different reference frames. The Oplus ($\oplus$) operation can be formally expressed as follows:

$$\begin{aligned}{}^A \mathbf{x}_C &= {}^A \mathbf{x}_B \oplus {}^B \mathbf{x}_C \\ &= \begin{bmatrix} {}^A x_B \\ {}^A y_B \\ {}^A \psi_B \end{bmatrix} \oplus \begin{bmatrix} {}^B x_C \\ {}^B y_C \\ {}^B \psi_C \end{bmatrix} \\ &= \begin{bmatrix} {}^A x_B + {}^B x_C \cos({}^A \psi_B) - {}^B y_C \sin({}^A \psi_B) \\ {}^A y_B + {}^B x_C \sin({}^A \psi_B) + {}^B y_C \cos({}^A \psi_B) \\ {}^A \psi_B + {}^B \psi_C \end{bmatrix}\end{aligned}\tag{2}$$

Where ${}^A\mathbf{x}_C$ is the state vector from $\{A\}$ to C express in frame $\{A\}$, ${}^A\mathbf{x}_B$ is the state vector from $\{A\}$ to B express in frame $\{A\}$, and ${}^B\mathbf{x}_C$ is the robot pose vector from $\{B\}$ to C express in frame $\{B\}$. In the other hand, Ominus ($\ominus$) operation as follow

$${}^B\mathbf{x}_A = \ominus {}^A\mathbf{x}_B = \begin{bmatrix} -{}^Ax_B \cos({}^A\psi_B) - {}^Ay_B \sin({}^A\psi_B) \\ {}^Ax_B \sin({}^A\psi_B) - {}^Ay_B \cos({}^A\psi_B) \\ -{}^Az_B \\ -{}^A\psi_B \end{bmatrix} \quad (3)$$

Where ${}^B\mathbf{x}_A$ is the pose vector from $\{B\}$ to $\{A\}$ express in $\{B\}$ and ${}^A\mathbf{x}_B$ is the pose vector from $\{A\}$ to $\{B\}$ express in $\{A\}$.

2.1 Differential Drive Robot Simulation

Moving on to the differential drive simulation robot implementation, we derive a class called **DifferentialDriveSimulatedRobot** from **SimulatedRobot** class whose template was provided to us. We override the **fs** method of the super class simulation using the forward kinematics equations with constant velocity assumption as follow,

$$\mathbf{x}_{s_k} = \begin{bmatrix} {}^N\boldsymbol{\eta}_{s_k} \\ {}^B\boldsymbol{\nu}_{s_k} \end{bmatrix} = f(\mathbf{x}_{s_{k-1}}, \mathbf{u}_{s_k}, \mathbf{w}_{s_k}) \quad (4)$$

$$= \begin{bmatrix} {}^N\boldsymbol{\eta}_{s_{k-1}} \oplus ({}^B\boldsymbol{\nu}_{s_{k-1}}\Delta t + \frac{1}{2}\mathbf{w}_{s_k}\Delta t^2) \\ {}^B\boldsymbol{\nu}_{s_{k-1}} + \mathbf{K}({}^B\boldsymbol{\nu}_d - {}^B\boldsymbol{\nu}_{s_{k-1}}) + \mathbf{w}_{s_k}\Delta t \end{bmatrix} \quad (5)$$

$$\mathbf{u}_k = [u_d \quad r_d]^\top, \quad (6)$$

$${}^B\boldsymbol{\nu}_d = [u_d \quad 0 \quad r_d]^\top \quad (7)$$

$$\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q}_w), \quad (8)$$

$$\mathbf{Q} = \begin{bmatrix} \sigma_u^2 & 0 & 0 \\ 0 & \sigma_v^2 & 0 \\ 0 & 0 & \sigma_r^2 \end{bmatrix} \quad (9)$$

Where, $\mathbf{K} = \text{diag}(k_1, k_2, k_3)$, $k_i > 0$ is a diagonal matrix containing the gains used to drive the simulated velocity towards the desired input velocity. $\mathbf{u}_k$ is input vector of the robot consist of the desired linear velocity along x-axis (u_d), linear velocity along y-axis (v_d) and angular velocity about z-axis (r_d). ${}^B\boldsymbol{\nu}_d$ is the desired velocity vector of the robot. Last, $\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q}_w)$ is the zero mean gaussian noise vector for the forward kinematics with constant velocity assumption, or can be implied as a acceleration. The implement those equations in code where also utilize the previously define **Pose3D** class and it's methods in estimating the next pose for the simulation given a velocity control command. We then use the **PlotTrajectory** function defined inside the **Localization** class to verify the simulation

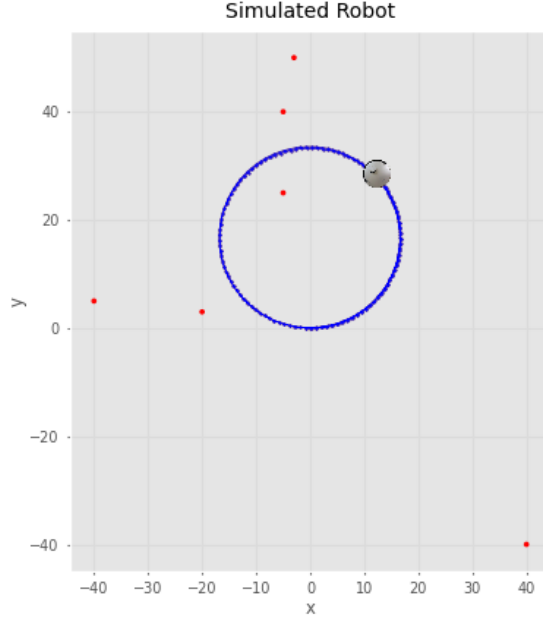


Figure 1: Simulation of the Differential Drive Robot

of the differential drive robot. In the resulting plot, the robot’s state is set to the simulated state, enabling visualization of the simulated robot’s trajectory as shown in Figure 1.

As shown in the Figure 1 trajectory of the simulated robot in an environment with acceleration noise are shown. The blue curve shows the robot’s noisy path, while red dots indicate static landmarks in the workspace. Acceleration noise results in small deviations from the ideal circular trajectory, illustrating the impact of motion uncertainty on robot navigation.

Furthermore, to ensure a comprehensive simulation of the robotic system, the wheel encoder sensor—which provides velocity measurements—was also simulated. This enables realistic modeling of the robot’s perception of motion based on encoder feedback, thereby improving the fidelity of the overall simulation.

2.2 Encoder Sensor Simulation

The simulation of encoders implemented in **ReadEncoders** method of the **DifferentialDriveSimulatedRobot**, where the sensors are provide only the number of pulse of the encoder with some sensors noise. To implement the encoder, the simulated linear and angular velocity are mapped to the encoder pulse by the following equation.

$$v_{r,s_k} = {}^B u_{s_k} + \left(\frac{{}^B r_{s_k} \cdot L}{2} \right) \quad (10)$$

$$v_{l,s_k} = {}^B u_{s_k} - \left(\frac{{}^B r_{s_k} \cdot L}{2} \right) \quad (11)$$

The equation 14 using to convert the velocity of the robot to the wheel velocity. Where, v_{r,s_k} and v_{l,s_k} are linear velocity of the right and left wheel, respectively, and $L = 0.5$ is the distance between right and left wheel. Then, pulse of the encoders are calculated by the velocity of the wheels by using follow equation

$$\mathbf{z}_{s_k} = \begin{bmatrix} n_{r,k} \\ n_{l,k} \end{bmatrix} \quad (12)$$

$$n_{r,k} = \frac{v_{r,s_k} \Delta t}{2\pi R_w} \cdot n_{2\pi} + \mathbf{v}_{r,s_k} \quad (13)$$

$$n_{l,k} = \frac{v_{l,s_k} \Delta t}{2\pi R_w} \cdot n_{2\pi} + \mathbf{v}_{l,s_k} \quad (14)$$

Where, $\Delta t = 1$ is the sampling interval, $R_w = 0.1$ is the wheel radius, $n_{2\pi} = 4096$ is number of pulse for each turn of the wheel, and $n_{r,k}$ and $n_{l,k}$ are number of pulse for each encoder with sensors noise. The noise for this sensor can be modeled as a vector of the random variable $\mathbf{v}_{s_k} \sim \mathcal{N}(0, \mathbf{R}_{s_k})$ which $\mathbf{v}_{s_k} = [v_{r,s_k}, v_{l,s_k}]^\top$ and $\mathbf{R}_{s_k} = \text{diag}(\sigma_L^2, \sigma_R^2)$.

2.3 System diagram of Differential Drive Robot Simulation

In summary, differential drive robot simulation system can be illustrate as in the Figure 2

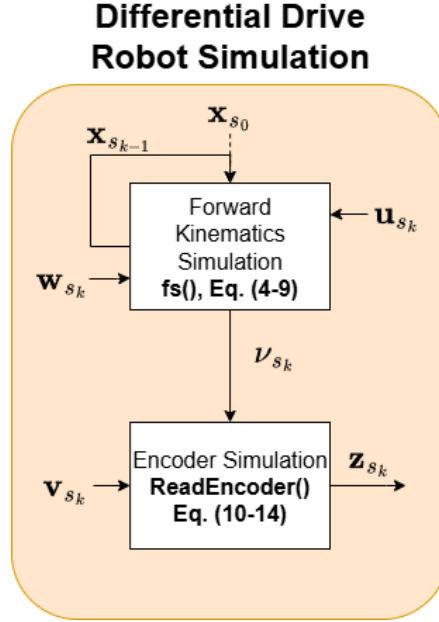


Figure 2: System diagram of Differential Drive Robot Simulation

3 Dead-Reckoning

The Dead-Reckoning localization technique estimates the current pose of a mobile robot by iteratively updating its position and orientation based solely on its preceding state, as well

as measurements of velocity and heading. The localization process generally comprises two steps: acquiring sensor input (**GetInput**) and estimating the robot's pose (**Localize**). In this laboratory exercise, encoder measurements from the robot's wheels serve as the input ($\mathbf{u}_k$) for the localization loop. During the localization step, these encoder readings are utilized to compute the robot's linear and angular velocities ($\hat{\mathbf{v}}_k$) follow by the odometry calculation which can be found in chapter 1 of [1].

3.1 Convert Encoder Sensors Output to Velocity

The sensor output that acquired from the robot simulation ($\mathbf{z}_{s_k}$) is considered as a input vector of the robot localization algorithm ($\mathbf{u}_k$). Then, the input vector are convert to the velocity of the robot by following steps.

First, the number of encoder pulse from both sensors are used to calculated the linear velocity of the wheels using the invert function of the sensors simulation .

$$\begin{bmatrix} n_{L,k} \\ n_{R,k} \end{bmatrix} = \mathbf{u}_k \quad (15)$$

$$\begin{bmatrix} v_L \\ v_R \end{bmatrix} = \frac{2\pi R}{n_{2\pi} \Delta t} \cdot \begin{bmatrix} n_{L,k} \\ n_{R,k} \end{bmatrix} \quad (16)$$

Next, linear velocity and angular velocity are calculated using the linear velocity of the left and right wheel.

$${}^B u_k = \frac{v_R + v_L}{2} \quad (17)$$

$${}^B r_k = \frac{v_R - v_L}{L} \quad (18)$$

$${}^B \hat{\mathbf{v}}_k = \begin{bmatrix} {}^B u_k \\ 0 \\ {}^B r_k \end{bmatrix} \quad (19)$$

3.2 Estimate Next Robot State

Subsequently, the estimated pose ($\hat{\eta}_k$) is updated using the computed velocities via the forward kinematic model, following an approach analogous to Equation 4.

$$\hat{\mathbf{x}}_k = \begin{bmatrix} {}^N \hat{\eta}_k \\ {}^B \hat{\mathbf{v}}_k \end{bmatrix} = \begin{bmatrix} {}^N \hat{\eta}_{k-1} \oplus ({}^B \hat{\mathbf{v}}_k \Delta t) \\ {}^B \hat{\mathbf{v}}_k \end{bmatrix} \quad (20)$$

However, In general differences in the sensor sampling rate and localization update frequency can impact system performance. If the sensor reading rate exceeds the localization update rate, the discrepancy can be managed by selectively skipping sensor readings. Conversely, if the sensor reading rate is lower than the localization update rate, the localization algorithm must rely on previously acquired sensor data, which may increase the accumulation of estimation errors.

3.3 System diagram of Dead-Reckoning localization

In summary, Dead-Reckoning localization with the simulation system can be illustrate as in the Figure 3

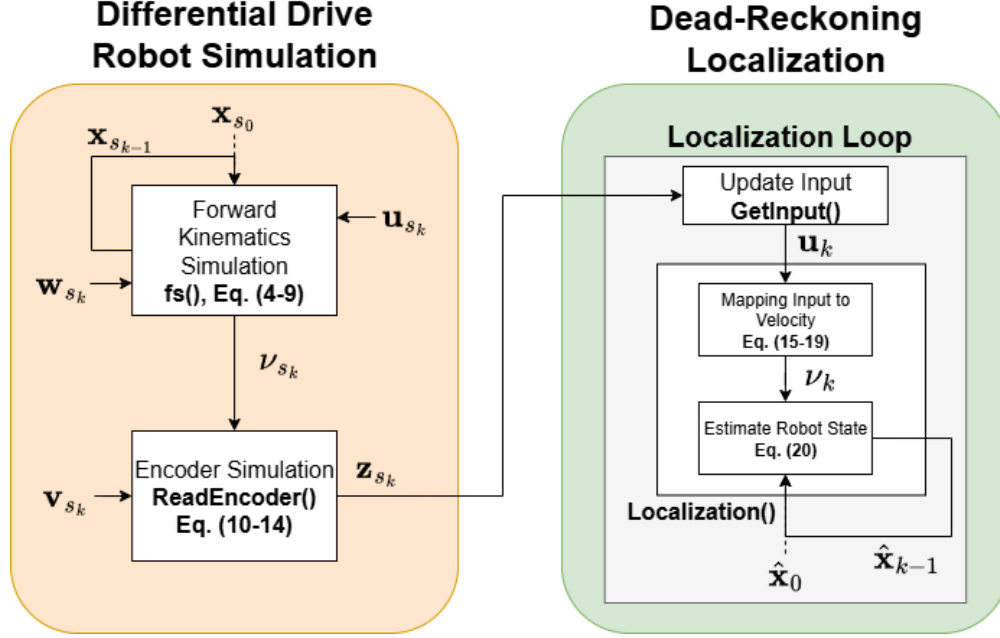


Figure 3: System diagram of Dead-Reckoning localization

3.4 Dead-Reckoning result

By continuously executing the localization loop to iteratively update the robot's pose, the resulting trajectory can be visualized as illustrated in Figure 4.

The Comparison of the simulated robot trajectory and dead-reckoning localization are plotted. The blue curve represents the ground truth trajectory generated by the simulated robot, while the orange curve depicts the path estimated through dead-reckoning. Red dots indicate fixed landmarks in the environment. The divergence between the simulated and estimated paths illustrates the accumulation of localization errors in dead-reckoning over time.

4 Issues

Several challenges were encountered during the course of this lab. Firstly, the installation of the required software package, `roboticstoolbox-python`, proved to be difficult due to dependencies that are not managed through the standard Python package manager (PIP). This issue could be mitigated in future labs by providing a comprehensive installation guide detailing all necessary software and dependencies, ensuring a smoother setup process.

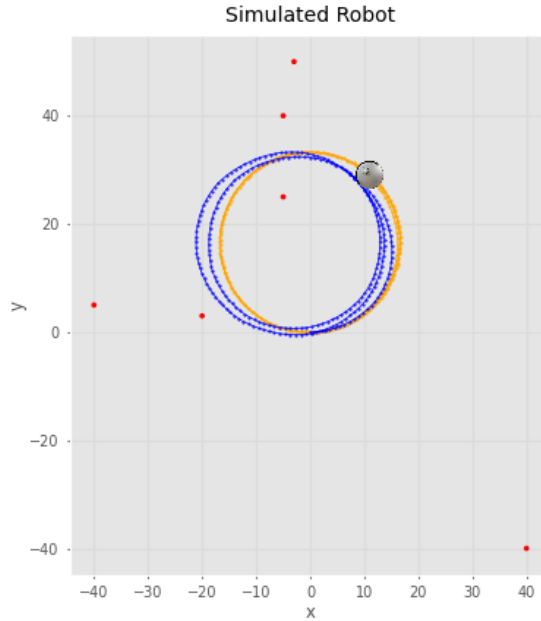


Figure 4: Trajectory Comparison: Simulated Robot vs. Dead-Reckoning Localization

Secondly, discrepancies were noted between the number of states and inputs presented in the reference materials, such as textbooks and lecture slides. This inconsistency led to some confusion during the development phase.

5 Conclusions

This lab exercise focused on developing a simulation framework for a differential drive mobile robot and implementing a dead-reckoning localization algorithm. The simulation incorporated the robot's forward kinematics with realistic sensor modeling, including acceleration noise and encoder feedback. Using the compounding pose operations, the robot's trajectory was estimated based on velocity control inputs, allowing for visualization and comparison of the simulated ground truth and dead-reckoning localization outputs. The dead-reckoning method utilized wheel encoder data to estimate linear and angular velocities, serving as the basis for iterative pose estimation through forward kinematics.

The results demonstrated the fundamental operation and limitations of dead-reckoning localization. While it is effective for estimating robot pose using incremental motion data, sensor noise and discrepancies in measurement update rates introduced errors that accumulated over time. The comparison between the simulated and dead-reckoning trajectories highlighted the divergence caused by sensor uncertainty. This lab reinforced the importance of accurate sensor modeling and the challenges inherent in dead-reckoning for mobile robot localization, offering practical experience in simulation, coding of robot kinematics, and state estimation algorithms.

References

- [1] Pere Ridao. "Probabilistic Robot Localization & Mapping". Unpublished manuscript. 2025